

构造与非传统题选讲

Harry27182

目录

- 1 2025 ICPC 网络赛 (II) J.Reconstruct the tree
- 2 2025 ICPC 网络赛 (I) F.Robot
- 3 2025 ICPC Shenyang C. Buggy Painting Software II
- 4 2025 ICPC Xi'an D. Directed Acyclic Graph
- 5 2025 ICPC Shenyang L. Leo

- 1 2025 ICPC 网络赛 (II) J.Reconstruct the tree
- 2 2025 ICPC 网络赛 (I) F.Robot
- 3 2025 ICPC Shenyang C. Buggy Painting Software II
- 4 2025 ICPC Xi'an D. Directed Acyclic Graph
- 5 2025 ICPC Shenyang L. Leo

Statements

有一个 n 个点的树，并给定了 m 个点对 (u_i, v_i) ，保证所有在树上路径长度等于直径长度的点对 (u, v) 恰好为给定集合。构造一棵满足条件的树或报告不存在。

$n, m \leq 2 \times 10^5$ 。

Solutions

- 我们根据和哪些点存在连边可以将 n 个点可以分成若干个集合 $S_1 \dots S_k$ 。

Solutions

- 我们根据和哪些点存在连边可以将 n 个点可以分成若干个集合 $S_1...S_k$ 。
- 容易发现，有解的必要条件为不同集合的点之间都存在连边，不在任何集合内的点被称为特殊点。

Solutions

- 我们根据和哪些点存在连边可以将 n 个点可以分成若干个集合 $S_1 \dots S_k$ 。
- 容易发现，有解的必要条件为不同集合的点之间都存在连边，不在任何集合内的点被称为特殊点。
- 如果 $k = 2$ ，那么可以直接把所有特殊点挂一条链然后两个集合的点分别挂在链的两侧。此时需要 2 个特殊点。

Solutions

- 如果 $k > 2$, 那么就需要一个类菊花的东西。我们将一个特殊点作为根, 然后再用 k 个特殊点分别作为 k 个集合对应子树的根。

Solutions

- 如果 $k > 2$, 那么就需要一个类菊花的东西。我们将一个特殊点作为根, 然后再用 k 个特殊点分别作为 k 个集合对应子树的根。
- 然后将每个集合的元素分别挂在这 k 个子树的根上。也就是总共需要 $k + 1$ 个特殊点。

Solutions

- 如果 $k > 2$, 那么就需要一个类菊花的东西。我们将一个特殊点作为根, 然后再用 k 个特殊点分别作为 k 个集合对应子树的根。
- 然后将每个集合的元素分别挂在这 k 个子树的根上。也就是总共需要 $k + 1$ 个特殊点。
- 如果直接这么写代码会喜提一个罚时。

Solutions

- 如果 $k > 2$, 那么就需要一个类菊花的東西。我們將一個特殊點作為根, 然後再用 k 個特殊點分別作為 k 個集合對應子樹的根。
- 然後將每個集合的元素分別挂在這 k 個子樹的根上。也就是總共需要 $k + 1$ 個特殊點。
- 如果直接這麼寫代碼會喜提一個罰時。
- 注意到有一種特殊情況, 如果有 $k = n - 1$, 那麼這個圖實際上就是一個菊花, 需要特判這個特殊情況。

Solutions

- 如果 $k > 2$, 那么就需要一个类菊花的東西。我們將一個特殊點作為根, 然後再用 k 個特殊點分別作為 k 個集合對應子樹的根。
- 然後將每個集合的元素分別挂在這 k 個子樹的根上。也就是總共需要 $k + 1$ 個特殊點。
- 如果直接這麼寫代碼會喜提一個罰時。
- 注意到有一種特殊情況, 如果有 $k = n - 1$, 那麼這個圖實際上就是一個菊花, 需要特判這個特殊情況。
- 然後記得特判一下一些平凡的 corner case 就可以通過了。

- 1 2025 ICPC 网络赛 (II) J.Reconstruct the tree
- 2 2025 ICPC 网络赛 (I) F.Robot
- 3 2025 ICPC Shenyang C. Buggy Painting Software II
- 4 2025 ICPC Xi'an D. Directed Acyclic Graph
- 5 2025 ICPC Shenyang L. Leo

Statements

一个机器人在平面直角坐标系的第一象限移动。最初，机器人位于网格坐标 (s_x, s_y) 处。所有网格单元初始状态均为白色。游戏按离散的时间步进行，最多持续 $T = 1000$ 个时间步。在每个时间步中，按以下顺序发生：

- ① 你的移动：选择一个整数坐标 (x_m, y_m) ，使得 $1 \leq x_m \leq T$ 且 $1 \leq y_m \leq T$ 。然后你将该单元格标记为黑色。
- ② 机器人的移动：机器人当前位于位置 (r_x, r_y) ，它会观察网格（包括你刚刚标记为黑色的单元格）。它知道所有当前黑色格的位置。然后它移动到一个新位置 (n_x, n_y) ，其中 $|n_x - r_x| \leq 1$ 且 $|n_y - r_y| \leq 1$ ，且 $(n_x, n_y) \neq (r_x, r_y)$ 。
- ③ 结果检查：在机器人移动到 (n_x, n_y) 之后，系统会检查单元格 (n_x, n_y) 是否为黑色。如果是黑色机器人爆炸，否则进入下一个时间步。

你的目标是使机器人在 T 个时间步之内爆炸。保证 $1 \leq s_x, s_y \leq 20$ 。

Solutions

- 我们先来思考一个简化的问题，如果我们能够把机器人限制在一个大小为 $D \times D$ 的正方形里，接下来应该怎么做？

Solutions

- 我们先来思考一个简化的问题，如果我们能够把机器人限制在一个大小为 $D \times D$ 的正方形里，接下来应该怎么做？
- 我们只需要交替按照 x 方向和 y 方向将矩形划分成两半，经过 $O(\log D)$ 轮即可让机器人爆炸。

Solutions

- 我们先来思考一个简化的问题，如果我们能够把机器人限制在一个大小为 $D \times D$ 的正方形里，接下来应该怎么做？
- 我们只需要交替按照 x 方向和 y 方向将矩形划分成两半，经过 $O(\log D)$ 轮即可让机器人爆炸。
- 等比数列求和可以得到这部分操作次数不超过 $3D$ 。

Solutions

- 我们先来思考一个简化的问题，如果我们能够把机器人限制在一个大小为 $D \times D$ 的正方形里，接下来应该怎么做？
- 我们只需要交替按照 x 方向和 y 方向将矩形划分成两半，经过 $O(\log D)$ 轮即可让机器人爆炸。
- 等比数列求和可以得到这部分操作次数不超过 $3D$ 。
- 下面问题在于我们如何将机器人限制在 $D \times D$ 的矩形内。

Solutions

- 第一想法应该是强行去画上上边界和右边界，但是这样需要 $2D$ 步，而机器人只需要 D 步就可以离开这个区域。

Solutions

- 第一想法应该是强行去画上上边界和右边界，但是这样需要 $2D$ 步，而机器人只需要 D 步就可以离开这个区域。
- 我们考虑退而求其次，先按照画两个空一个的方法画上上边界，这样只用了 $\frac{2}{3}D$ 步。

Solutions

- 第一想法应该是强行去画上上边界和右边界，但是这样需要 $2D$ 步，而机器人只需要 D 步就可以离开这个区域。
- 我们考虑退而求其次，先按照画两个空一个的方法画上上边界，这样只用了 $\frac{2}{3}D$ 步。
- 此时机器人还需要 $\frac{1}{3}D$ 步才可以从右边界离开这个区域。

Solutions

- 第一想法应该是强行去画上上边界和右边界，但是这样需要 $2D$ 步，而机器人只需要 D 步就可以离开这个区域。
- 我们考虑退而求其次，先按照画两个空一个的方法画上上边界，这样只用了 $\frac{2}{3}D$ 步。
- 此时机器人还需要 $\frac{1}{3}D$ 步才可以从右边界离开这个区域。
- 这时候我们画两个空一个的好处就体现到了，如果机器人想从上边界逃跑，它一次能够出去的位置至多有一个，所以在机器人即将从上边加逃跑的时候画上对应位置即可。

Solutions

- 第一想法应该是强行去画上上边界和右边界，但是这样需要 $2D$ 步，而机器人只需要 D 步就可以离开这个区域。
- 我们考虑退而求其次，先按照画两个空一个的方法画上上边界，这样只用了 $\frac{2}{3}D$ 步。
- 此时机器人还需要 $\frac{1}{3}D$ 步才可以从右边界离开这个区域。
- 这时候我们画两个空一个的好处就体现到了，如果机器人想从上边界逃跑，它一次能够出去的位置至多有一个，所以在机器人即将从上边加逃跑的时候画上对应位置即可。
- 否则我们贪心的画上右边界上与机器人曼哈顿距离最小的位置，容易发现这样是可以让无法机器人从右边界离开的。

Solutions

- 第一想法应该是强行去画上上边界和右边界，但是这样需要 $2D$ 步，而机器人只需要 D 步就可以离开这个区域。
- 我们考虑退而求其次，先按照画两个空一个的方法画上上边界，这样只用了 $\frac{2}{3}D$ 步。
- 此时机器人还需要 $\frac{1}{3}D$ 步才可以从右边界离开这个区域。
- 这时候我们画两个空一个的好处就体现到了，如果机器人想从上边界逃跑，它一次能够出去的位置至多有一个，所以在机器人即将从上边加逃跑的时候画上对应位置即可。
- 否则我们贪心的画上右边界上与机器人曼哈顿距离最小的位置，容易发现这样是可以让无法机器人从右边界离开的。
- 最后补全所有上边界空着的位置即可，共需要 $2D$ 的操作次数。

Solutions

- 第一想法应该是强行去画上上边界和右边界，但是这样需要 $2D$ 步，而机器人只需要 D 步就可以离开这个区域。
- 我们考虑退而求其次，先按照画两个空一个的方法画上上边界，这样只用了 $\frac{2}{3}D$ 步。
- 此时机器人还需要 $\frac{1}{3}D$ 步才可以从右边界离开这个区域。
- 这时候我们画两个空一个的好处就体现到了，如果机器人想从上边界逃跑，它一次能够出去的位置至多有一个，所以在机器人即将从上边加逃跑的时候画上对应位置即可。
- 否则我们贪心的画上右边界上与机器人曼哈顿距离最小的位置，容易发现这样是可以让无法机器人从右边界离开的。
- 最后补全所有上边界空着的位置即可，共需要 $2D$ 的操作次数。
- 总操作次数为 $5D - O(1)$ ，取 $D = 200$ 即可通过。

- 1 2025 ICPC 网络赛 (II) J.Reconstruct the tree
- 2 2025 ICPC 网络赛 (I) F.Robot
- 3 2025 ICPC Shenyang C. Buggy Painting Software II
- 4 2025 ICPC Xi'an D. Directed Acyclic Graph
- 5 2025 ICPC Shenyang L. Leo

Statements

通信题。第一次运行需要构造 n 个长度为 $3m$ 的序列，每个序列中 1 到 m 的整数恰好出现三次，要求第 i 个序列加密了 x_i 这个数 (x_i 的范围也是 1 到 m)。然后 SPJ 会独立地将每个序列用某个二划分 (S_0, S_1) 映射成二进制序列，第二次运行需要从这些二进制序列中解密出每个 x_i 。

$2 \leq n, m \leq 5 \times 10^5, n \times m \leq 10^6$ 。

Solutions

- 很自然的一个想法是首先将 $1 \sim m$ 放在序列的开头，这样可以用 n 个位置得到具体的 (S_0, S_1) 。

Solutions

- 很自然的一个想法是首先将 $1 \sim m$ 放在序列的开头，这样可以用 n 个位置得到具体的 (S_0, S_1) 。
- 接下来我们需要让剩下的 $2m$ 个位置的区别尽可能大，让 SPJ 不能构造一种方式使得不同的 x_i 能对应相同的 01 序列。

Solutions

- 很自然的一个想法是首先将 $1 \sim m$ 放在序列的开头，这样可以用 n 个位置得到具体的 (S_0, S_1) 。
- 接下来我们需要让剩下的 $2m$ 个位置的区别尽可能大，让 SPJ 不能构造一种方式使得不同的 x_i 能对应相同的 01 序列。
- 所以加密 x_i 时尝试将 $1 \sim m$ 循环移位 x_i 位放在开头的 $1 \sim m$ 后面。由裴蜀定理知，当 m 为质数时， $x_i \neq x_j$ 不会被映射到同个二进制序列，否则 $\gcd(|x_i - x_j|, m) = 1$ 可以推出二进制序列要么全 0 要么全 1，即反证。

Solutions

- 很自然的一个想法是首先将 $1 \sim m$ 放在序列的开头，这样可以用 n 个位置得到具体的 (S_0, S_1) 。
- 接下来我们需要让剩下的 $2m$ 个位置的区别尽可能大，让 SPJ 不能构造一种方式使得不同的 x_i 能对应相同的 01 序列。
- 所以加密 x_i 时尝试将 $1 \sim m$ 循环移位 x_i 位放在开头的 $1 \sim m$ 后面。由裴蜀定理知，当 m 为质数时， $x_i \neq x_j$ 不会被映射到同个二进制序列，否则 $\gcd(|x_i - x_j|, m) = 1$ 可以推出二进制序列要么全 0 要么全 1，即反证。
- 上述做法在 m 为合数时仍可被攻击。幸运的是，剩余位置有 $2m$ 个而非 m 个，这启发我们耗费更多位置来构造等价于质数的加密方式。

Solutions

- 根据伯特兰-切比雪夫定理, m 到 $2m$ 之间必然存在一个质数 p 。

Solutions

- 根据伯特兰-切比雪夫定理, m 到 $2m$ 之间必然存在一个质数 p 。
- 从 $1, 2 \cdots m, 1, 2 \cdots m, 1, 2 \cdots m$ 结构出发, 前 m 个位置仍然用于得到 (S_0, S_1) , 接下来 p 个位置 $1, 2 \cdots m, [1, 2 \cdots m, 1, 2, \cdots p - m], p - m + 1 \cdots m$ 用于循环移位 x_i 位。

Solutions

- 根据伯特兰-切比雪夫定理, m 到 $2m$ 之间必然存在一个质数 p 。
- 从 $1, 2 \cdots m, 1, 2 \cdots m, 1, 2 \cdots m$ 结构出发, 前 m 个位置仍然用于得到 (S_0, S_1) , 接下来 p 个位置
 $1, 2 \cdots m, [1, 2 \cdots m, 1, 2, \cdots p - m], p - m + 1 \cdots m$ 用于循环移位 x_i 位。
- 解密时, 用 (S_0, S_1) 推出 $1, 2 \cdots m, 1, 2, \cdots p - m$ 循环移位前的二进制序列, 要知道循环移位了几位是一个经典问题, 可以通过倍长后 KMP 算法解决。

Solutions

- 根据伯特兰-切比雪夫定理, m 到 $2m$ 之间必然存在一个质数 p 。
- 从 $1, 2 \cdots m, 1, 2 \cdots m, 1, 2 \cdots m$ 结构出发, 前 m 个位置仍然用于得到 (S_0, S_1) , 接下来 p 个位置
 $1, 2 \cdots m, [1, 2 \cdots m, 1, 2, \cdots p - m], p - m + 1 \cdots m$ 用于循环移位 x_i 位。
- 解密时, 用 (S_0, S_1) 推出 $1, 2 \cdots m, 1, 2, \cdots p - m$ 循环移位前的二进制序列, 要知道循环移位了几位是一个经典问题, 可以通过倍长后 KMP 算法解决。
- 单次加密、解密的复杂度是 $O(m)$, 总时间复杂度为 $O(nm)$ 。

- 1 2025 ICPC 网络赛 (II) J.Reconstruct the tree
- 2 2025 ICPC 网络赛 (I) F.Robot
- 3 2025 ICPC Shenyang C. Buggy Painting Software II
- 4 2025 ICPC Xi'an D. Directed Acyclic Graph
- 5 2025 ICPC Shenyang L. Leo

Statements

你需要构造一个包含 n 个点和 m 条边的有向无环图。其中定义 u 可达 v 为存在一条 $u \rightarrow v$ 的路径。定义 $f(A)$ 表示 A 集合内所有点的共同可达点。

你构造的图需要满足以下条件：1 可达所有点；存在 k 个两两不同的非空点集 $S_1 \sim S_k$ ，满足 $f(S_1) \sim f(S_k)$ 两两不同。

$n = 100, m = 128, k = 16000$ 。

Solutions

- 为了满足 1 可达所有点的条件，我们必须构造一棵以 1 为根的外向生成树，这样就需要用掉 $n - 1$ 条边。

Solutions

- 为了满足 1 可达所有点的条件，我们必须构造一棵以 1 为根的外向生成树，这样就需要用掉 $n - 1$ 条边。
- 也就是说，我们可以用于满足第二个条件的额外边只有 $m - n + 1 = 29$ 条，容易发现这个限制实际上是更紧的。问题转化为如何用 29 条边构造出 k 个满足条件的集合。

Solutions

- 为了满足 1 可达所有点的条件，我们必须构造一棵以 1 为根的外向生成树，这样就需要用掉 $n - 1$ 条边。
- 也就是说，我们可以用于满足第二个条件的额外边只有 $m - n + 1 = 29$ 条，容易发现这个限制实际上是更紧的。问题转化为如何用 29 条边构造出 k 个满足条件的集合。
- 首先给出一种较为容易的构造：取一个正整数 w ，对于一行结点 $a_1, a_2, \dots, a_w$ ，构造另一行结点 $b_1, b_2, \dots, b_w$ ，使得 b_i 能够到达所有满足 $j \neq i$ 的 a_j 。这样可以构造出至少 2^w 个不同的可达点集，因为对于每个 a_i ，都可以用 b_i 是否在所选的集合中来自由控制 a_i 是否在可达点集中。

Solutions

- 为了满足 1 可达所有点的条件，我们必须构造一棵以 1 为根的外向生成树，这样就需要用掉 $n - 1$ 条边。
- 也就是说，我们可以用于满足第二个条件的额外边只有 $m - n + 1 = 29$ 条，容易发现这个限制实际上是更紧的。问题转化为如何用 29 条边构造出 k 个满足条件的集合。
- 首先给出一种较为容易的构造：取一个正整数 w ，对于一行结点 $a_1, a_2, \dots, a_w$ ，构造另一行结点 $b_1, b_2, \dots, b_w$ ，使得 b_i 能够到达所有满足 $j \neq i$ 的 a_j 。这样可以构造出至少 2^w 个不同的可达点集，因为对于每个 a_i ，都可以用 b_i 是否在所选的集合中来自由控制 a_i 是否在可达点集中。
- 可以用前缀后缀优化建图，这部分的额外边数为 $3w$ ，也就是可以用 $3w$ 的边数做到 2^w 种本质不同的集合。大约需要 140 条边。

Solutions

- 考虑拓展上面的构造方案，我们构造两个 d 行 w 列的点阵 $a_{i,j}$ 和 $b_{i,j}$ ，其中每个 $b_{i,j}$ 都可达 a 中所有点去掉某一系列的一个前缀。

Solutions

- 考虑拓展上面的构造方案，我们构造两个 d 行 w 列的点阵 $a_{i,j}$ 和 $b_{i,j}$ ，其中每个 $b_{i,j}$ 都可达 a 中所有点去掉某一系列的一个前缀。
- 同样使用前后缀优化建图可以做到使用 $(d+2)w$ 条额外边构造出 $(d+1)^w$ 个不同的集合。

Solutions

- 考虑拓展上面的构造方案，我们构造两个 d 行 w 列的点阵 $a_{i,j}$ 和 $b_{i,j}$ ，其中每个 $b_{i,j}$ 都可达 a 中所有点去掉某一系列的一个前缀。
- 同样使用前后缀优化建图可以做到使用 $(d+2)w$ 条额外边构造出 $(d+1)^w$ 个不同的集合。
- 具体的 $(d+1)^w$ 个不同的集合就是每一列都可以选择可达 $0 \sim d$ 个。

Solutions

- 考虑拓展上面的构造方案，我们构造两个 d 行 w 列的点阵 $a_{i,j}$ 和 $b_{i,j}$ ，其中每个 $b_{i,j}$ 都可达 a 中所有点去掉某一系列的一个前缀。
- 同样使用前后缀优化建图可以做到使用 $(d+2)w$ 条额外边构造出 $(d+1)^w$ 个不同的集合。
- 具体的 $(d+1)^w$ 个不同的集合就是每一列都可以选择可达 $0 \sim d$ 个。
- 而 $(d+2)w$ 中有 dw 条边是每个 $b_{i,j}$ 需要两条出边，剩下的 $2w$ 条额外边用于每个前后缀上的点都需要两条出边。

Solutions

- 考虑拓展上面的构造方案，我们构造两个 d 行 w 列的点阵 $a_{i,j}$ 和 $b_{i,j}$ ，其中每个 $b_{i,j}$ 都可达 a 中所有点去掉某一系列的一个前缀。
- 同样使用前后缀优化建图可以做到使用 $(d+2)w$ 条额外边构造出 $(d+1)^w$ 个不同的集合。
- 具体的 $(d+1)^w$ 个不同的集合就是每一列都可以选择可达 $0 \sim d$ 个。
- 而 $(d+2)w$ 中有 dw 条边是每个 $b_{i,j}$ 需要两条出边，剩下的 $2w$ 条额外边用于每个前后缀上的点都需要两条出边。
- 取 $d=3$ 可以达到最优，用 $5w$ 条边达到 4^w 种方案。取 $w=7$ 并经过一些卡常即可通过。

- 1 2025 ICPC 网络赛 (II) J.Reconstruct the tree
- 2 2025 ICPC 网络赛 (I) F.Robot
- 3 2025 ICPC Shenyang C. Buggy Painting Software II
- 4 2025 ICPC Xi'an D. Directed Acyclic Graph
- 5 2025 ICPC Shenyang L. Leo

Statements

实现一个三色逻辑计算机，满足 n 个输入端从 $\{R, G, B, *\}$ 中取时，计算机总能算出从左到右遇到的第三种不同颜色。要求至多用 $6n$ 个元件。

其中有三色 AND 和 OR 操作，AND 操作当且仅当两个为同一种非 $*$ 的颜色时会为这种颜色，否则为 $*$ 。

OR 操作两个相同的仍然保持，如果一个为颜色另一个为 $*$ 那么返回那个颜色，如果两个不同颜色返回第三种颜色。

$3 \leq n \leq 10^5$ 。

Solutions

- 第一步：消除大部分 $*$ ，只允许 $*$ 出现在一小段后缀中。

Solutions

- 第一步：消除大部分 *，只允许 * 出现在一小段后缀中。
- 其动机是，AND 容易受到 * 干扰导致始终产生 * 而非预期之内的结果，于是我们希望每个 * 都能被就近的颜色取代。

Solutions

- 第一步：消除大部分 $*$ ，只允许 $*$ 出现在一小段后缀中。
- 其动机是，AND 容易受到 $*$ 干扰导致始终产生 $*$ 而非预期之内的结果，于是我们希望每个 $*$ 都能被就近的颜色取代。
- 不妨考虑让每个 $*$ 被其后最近的颜色取代（如果存在的话），这样做的好处是，输入端从左到右遇到的第一种颜色就在第一个位置，只要能设法找到从左到右首个异于该颜色的位置，OR 一下就是答案。

Solutions

- 第一步：消除大部分 $*$ ，只允许 $*$ 出现在一小段后缀中。
- 其动机是，AND 容易受到 $*$ 干扰导致始终产生 $*$ 而非预期之内的结果，于是我们希望每个 $*$ 都能被就近的颜色取代。
- 不妨考虑让每个 $*$ 被其后最近的颜色取代（如果存在的话），这样做的好处是，输入端从左到右遇到的第一种颜色就在第一个位置，只要能设法找到从左到右首个异于该颜色的位置，OR 一下就是答案。
- 不难发现 $x \text{ OR } y \text{ OR } y$ 是一个“二路优先选择门”，该元件可以在 x 非 $*$ 仍然返回 x ，而 x 为 $*$ 时返回 y 。

Solutions

- 第一步：消除大部分 $*$ ，只允许 $*$ 出现在一小段后缀中。
- 其动机是，AND 容易受到 $*$ 干扰导致始终产生 $*$ 而非预期之内的结果，于是我们希望每个 $*$ 都能被就近的颜色取代。
- 不妨考虑让每个 $*$ 被其后最近的颜色取代（如果存在的话），这样做的好处是，输入端从左到右遇到的第一种颜色就在第一个位置，只要能设法找到从左到右首个异于该颜色的位置，OR 一下就是答案。
- 不难发现 $x \text{ OR } y \text{ OR } y$ 是一个“二路优先选择门”，该元件可以在 x 非 $*$ 仍然返回 x ，而 x 为 $*$ 时返回 y 。
- 所以我们可以使用该元件倒着扫描输入端，通过“后缀优先选择”达成目的，我们设此时的序列为 $a_1, a_2, \dots, a_n$ 。这一步的操作次数为 $2n$ 。

Solutions

- 第二步：标识出从左到右遇到的第二种不同颜色的位置。

Solutions

- 第二步：标识出从左到右遇到的第二种不同颜色的位置。
- 延续第一步的思路，碍于黑盒机制我们不可能直接点对点地 OR 到第二种颜色，因为它可能在任何位置，我们应当侧重如何让它在任何位置时都能被标识，具体如下：

Solutions

- 第二步：标识出从左到右遇到的第二种不同颜色的位置。
- 延续第一步的思路，碍于黑盒机制我们不可能直接点对点地 OR 到第二种颜色，因为它可能在任何位置，我们应当侧重如何让它在任何位置时都能被标识，具体如下：
- 令 $f_i = f_{i-1} \text{ AND } a_i$ 和 $b_i = f_{i-1} \text{ OR } a_i$ ，显然 b 序列仅在从左到右遇到的第二种不同颜色的位置处和 a 序列不同。

Solutions

- 第三步：计算出从左到右遇到的第三种不同颜色。

Solutions

- 第三步：计算出从左到右遇到的第三种不同颜色。
- 从 a, b 序列只有一个位置不同出发，容易想到构造类似 $(a_1 - b_1) + (a_2 - b_2) + \cdots + (a_n - b_n)$ 的代数运算来抵消相同位置，而差异位置贡献出目标颜色。

Solutions

- 第三步：计算出从左到右遇到的第三种不同颜色。
- 从 a, b 序列只有一个位置不同出发，容易想到构造类似 $(a_1 - b_1) + (a_2 - b_2) + \cdots + (a_n - b_n)$ 的代数运算来抵消相同位置，而差异位置贡献出目标颜色。
- 而 OR 刚好具备优秀的代数性质，分别记从左到右遇到的第一、二、三种颜色为 $0, 1, 2$ ，则 $x \text{ OR } y \equiv -x - y \pmod{3}$ ，进而偶数项的 OR 链： $x_1 \text{ OR } x_2 \text{ OR } \cdots \equiv -x_1 - x_2 + x_3 - x_4 + \cdots \pmod{3}$ 。

Solutions

- 第三步：计算出从左到右遇到的第三种不同颜色。
- 从 a, b 序列只有一个位置不同出发，容易想到构造类似 $(a_1 - b_1) + (a_2 - b_2) + \cdots + (a_n - b_n)$ 的代数运算来抵消相同位置，而差异位置贡献出目标颜色。
- 而 OR 刚好具备优秀的代数性质，分别记从左到右遇到的第一、二、三种颜色为 0, 1, 2，则 $x \text{ OR } y \equiv -x - y \pmod{3}$ ，进而偶数项的 OR 链： $x_1 \text{ OR } x_2 \text{ OR } \cdots \equiv -x_1 - x_2 + x_3 - x_4 + \cdots \pmod{3}$ 。
- 同时结合 $a_1 = b_1 = 0$ ，我们可以得到 $a_1 \text{ OR } b_1 \text{ OR } a_2 \text{ OR } b_2 \text{ OR } \cdots \text{ OR } a_n \text{ OR } b_n$ 的代数结果在 mod3 意义下就是 $a_k - b_k \equiv 1 - 2 \equiv 2$ (k 是两序列的差异位置)，刚好是第三种颜色，并且即使 a, b 有等长的 * 后缀，该颜色结果显然会保留到最后。

Solutions

- 第三步：计算出从左到右遇到的第三种不同颜色。
- 从 a, b 序列只有一个位置不同出发，容易想到构造类似 $(a_1-b_1) + (a_2-b_2) + \cdots + (a_n-b_n)$ 的代数运算来抵消相同位置，而差异位置贡献出目标颜色。
- 而 OR 刚好具备优秀的代数性质，分别记从左到右遇到的第一、二、三种颜色为 $0, 1, 2$ ，则 $x \text{ OR } y \equiv -x-y \pmod{3}$ ，进而偶数项的 OR 链： $x_1 \text{ OR } x_2 \text{ OR } \cdots \equiv -x_1-x_2+x_3-x_4+\cdots \pmod{3}$ 。
- 同时结合 $a_1 = b_1 = 0$ ，我们可以得到 $a_1 \text{ OR } b_1 \text{ OR } a_2 \text{ OR } b_2 \text{ OR } \cdots \text{ OR } a_n \text{ OR } b_n$ 的代数结果在 $\text{mod}3$ 意义下就是 $a_k-b_k \equiv 1-2 \equiv 2$ (k 是两序列的差异位置)，刚好是第三种颜色，并且即使 a, b 有等长的 $*$ 后缀，该颜色结果显然会保留到最后。
- 三个步骤有机结合，成功构造了 $6n-O(1)$ 个元件的计算机。